

# Algorithms and Data Structures I.

## Sample First Test

**Note:** The sample test was created with examples from the practice class material. On the real test, if the task contains multiple *Options*, it will be chosen randomly.

### Task 1 (4 points):

Note: This task will be one of the following four simple sorting algorithms:

*Insertion, Maximum Selection,  
Bubble, Improved Bubble.*

Draw the structogram of the **Maximum Selection** sorting algorithm using arrays! Detail the running time for each line next to the structogram. What is the operation complexity of the algorithm?

### Task 2 (8 points):

Note: This task will be one of the following four simple sorting algorithms:

*Insertion, Maximum Selection,  
Bubble, Improved Bubble.*

Sort the array below using the **Insertion** sorting algorithm! Show the detailed operation of the algorithm in each iteration!

$\langle 5, 4, 2, 8, 3 \rangle$

### Task 3 (12 points):

Note: This task will be one of the following:

*Merge Sort,  
Quicksort.*

Sort the following array using the **Quicksort** algorithm (the version learned in class)!

a) Show in detail how the *partition*( $A, p, r$ ) algorithm works during the first call! Assume that the algorithm selects the element **14** as the pivot in the first call for the following array:

$\langle 12, 31, 23, 20, 14, 55, 11 \rangle$

b) Show how the subsequent calls proceed after the first one, and describe the result of each call by showing the current state of the array! The pivot element may be selected randomly in each call.

### Task 4 (12 points):

a) Write the (reversed) **Polish notation** of the following infix expression using a stack, according to the method learned in class:

$$x := b + 2 / (a - c) - (-f \wedge b \wedge 2 * g) / z + w$$

b) Evaluate the following reverse Polish (postfix) expressions using a stack, according to the method learned in class.  $a = 12$ ,  $b = 8$ ,  $c = 2$ ,  $d = 3$

$$a \ b + \ c \ / \ d \ 2 \wedge -$$

### Task 5 (14 points):

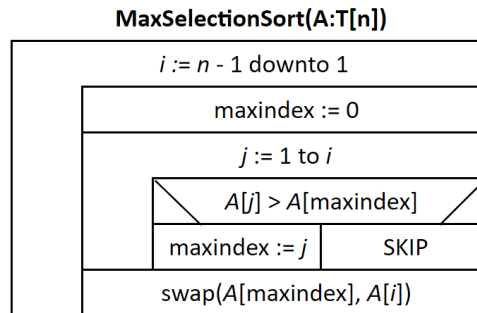
Let  $H1$  and  $H2$  be sorted H1Ls representing sets! Write the **Union**( $H1, H2 : E1^*$ ) algorithm returning the union of the two sets in  $H1$ , by unlinking/deleting elements from  $H2$ . **Do not allocate new elements!**  $MT(m, n) \in \Theta(m + n)$ , where  $|H1| = m$ , and  $|H2| = n$ .  $H1$  should remain sorted.

# Algorithms and Data Structures I.

## Sample Midterm Solutions

### Solution 1.

Illustrate the **maximum selection sort** algorithm's structogram using arrays.



Define swap separately how it works! (using temporal variable)

### Solution 2.

Sort the array below using the **insertion sort** algorithm. Demonstrate how the algorithm works in detail in every iteration.

$\langle 5, 4, 2, 8, 3 \rangle$

| Array:        | $\langle 5, 4, 2, 8, 3 \rangle$      | Comments                            |
|---------------|--------------------------------------|-------------------------------------|
| 1. iteration: | $\langle 5 \mid 4, 2, 8, 3 \rangle$  | 5 is sorted                         |
| 2. iteration: | $\langle 4, 5 \mid 2, 8, 3 \rangle$  | Inserting 4 before 5 to keep sorted |
| 3. iteration: | $\langle 2, 4, 5 \mid 8, 3 \rangle$  | ...                                 |
| 4. iteration: | $\langle 2, 4, 5, 8 \mid 3 \rangle$  | ...                                 |
| 5. iteration: | $\langle 2, 3, 4, 5, 8 \mid \rangle$ | ...                                 |
| Sorted array: | $\langle 2, 3, 4, 5, 8 \rangle$      |                                     |

### Solution 3.

Sort the array below with **quicksort** (the version learned in class).

a) Demonstrate how the  $partition(A, p, r)$  algorithm works in detail only for the first call. Let's assume that the algorithm chooses the element 14 as the pivot in the current array in the first call.

$\langle 12, 31, 23, 20, 14, 55, 11 \rangle$

| Array:                                       | $\langle 12, 31, 23, 20, 14, 55, 11 \rangle$                                                        |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Choosing the <b>PIVOT</b> item:              | $\langle 12, 31, 23, 20, \textcolor{brown}{14}, 55, 11 \rangle$                                     |
| Swap with the last element:                  | $\langle 12, 31, 23, 20, 11, 55, \textcolor{brown}{14} \rangle$                                     |
| Look for $A[i] > \textcolor{brown}{PIVOT}$ : | $\langle 12, \textcolor{blue}{31}, 23, 20, 11, 55, \textcolor{brown}{14} \rangle$                   |
| Set $j := i + 1$ :                           | $\langle 12, \textcolor{blue}{31}, \textcolor{red}{23}, 20, 11, 55, \textcolor{brown}{14} \rangle$  |
| Look for $A[j] < \textcolor{brown}{PIVOT}$ : | $\langle 12, \textcolor{blue}{31}, 23, 20, \textcolor{red}{11}, 55, \textcolor{brown}{14} \rangle$  |
| <b>SWAP</b> $A[i]$ and $A[j]$ :              | $\langle 12, \textcolor{blue}{11}, 23, 20, \textcolor{blue}{31}, 55, \textcolor{brown}{14} \rangle$ |
| Set $i := i + 1$ :                           | $\langle 12, 11, \textcolor{blue}{23}, 20, \textcolor{red}{31}, 55, \textcolor{brown}{14} \rangle$  |
| Continue $A[j] < \textcolor{brown}{PIVOT}$ : | $\langle 12, 11, \textcolor{blue}{23}, 20, \textcolor{red}{31}, 55, \textcolor{brown}{14} \rangle$  |
| $j$ reaches the <b>PIVOT</b> :               | $\langle 12, 11, \textcolor{blue}{23}, 20, 31, 55, \textcolor{brown}{14} \rangle$                   |
| <b>SWAP</b> $A[i]$ with the <b>PIVOT</b> :   | $\langle 12, 11, \textcolor{brown}{14}, 20, 31, 55, \textcolor{blue}{23} \rangle$                   |

$\langle A \text{ (small)} \rangle \textcolor{brown}{PIVOT} \langle B \text{ (big)} \rangle$  : |  $\langle 12, 11 \rangle \textcolor{brown}{14} \langle 20, 31, 55, 23 \rangle$

b) Demonstrate how the subsequent calls are executed after the first call and write down the result of each call by showing the values in the array. Pivot can be chosen randomly.

|    |    |    |    |    |    |    |   |   |
|----|----|----|----|----|----|----|---|---|
|    | 8  |    | 2  |    | 3  | 2  |   |   |
| 12 | 12 | 20 | 20 | 10 | 10 | 10 | 9 | 1 |
| a  | b  | +  | c  | /  | d  | 2  | ^ | - |

### Solution 5.

Let  $H1$  and  $H2$  be sorted H1Ls representing sets! Write the  $\text{Union}(H1, H2 : E1^*)$  algorithm returning the union of the two sets into  $H1$ , by unlinking/deleting elements from  $H2$ . **Do not allocate new elements!**  $MT(m, n) \in \Theta(m + n)$ , where  $|H1| = m$ , and  $|H2| = n$ .  $H1$  should remain sorted.

The following is an extended algorithm that does the Union with H1L and also leaves the intersection elements in the second list.

| unionIntersection( Hu, Hi : E1*)                                        |                                                                                                                                                                       |                                                                                                          |  |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--|
| qe:=Hu;    q:=Hu->next;    re:=Hi;    r:=Hi->next                       |                                                                                                                                                                       |                                                                                                          |  |
| q ≠ 0 ∧ r ≠ 0                                                           |                                                                                                                                                                       |                                                                                                          |  |
| <div>q-&gt;key &lt; r-&gt; key</div> <div>qe:=q<br/>q:=q-&gt;next</div> | <div>q-&gt;key &gt; r-&gt; key</div> <div>re-&gt;next:=r-&gt;next</div> <div>qe-&gt;next:=r</div> <div>r-&gt;next:=q</div> <div>qe:=r</div> <div>r:=re-&gt;next</div> | <div>q-&gt;key = r-&gt; key</div> <div>qe:= q<br/>q:=q-&gt;next</div> <div>re:=r<br/>r:=r-&gt;next</div> |  |
|                                                                         |                                                                                                                                                                       |                                                                                                          |  |
|                                                                         |                                                                                                                                                                       |                                                                                                          |  |
|                                                                         |                                                                                                                                                                       |                                                                                                          |  |
|                                                                         |                                                                                                                                                                       |                                                                                                          |  |
|                                                                         |                                                                                                                                                                       |                                                                                                          |  |
| r ≠ 0                                                                   |                                                                                                                                                                       |                                                                                                          |  |
| qe->next := r                                                           |                                                                                                                                                                       | skip                                                                                                     |  |
| re-> next:= 0                                                           |                                                                                                                                                                       |                                                                                                          |  |